Khmelnytskyi National University
Department of Information Technonogy

Report

Laboratory work №6

Discipline: “Programming”

Topic: “*Functions*”

Completed: 1st year student, group SE-26-1 Nykon Hrytsyshyn
Name, Surname

Checked: Valeriy Martynyuk
Name, Surname

Khmelnytskyi, 2026

Purpose: to learn means of describing user functions, to learn apply user functions.

Technical Environment: The project is implemented in accordance with the C11 standard and configured for cross-platform compilation using CMake and the Ninja build system. The application is built and verified using modern C compilers (GCC, Clang, and MSVC) with strict diagnostic flags enabled, ensuring high code quality, portability, and compatibility across operating systems.

Note: The software implementation is designed using a modular architecture with robust input validation and an interactive task execution engine. Detailed flowcharts illustrating the overall execution logic, along with full source code for all program components (tasks.h, matrix.h, utils.h, main.c, runner.c, matrix.c and utils.c), are provided in Appendices A…H.

In Table 1, general information about the tasks is provided.

Table 1 – Tasks overview

Task	What it does
1	Fills an n x t array with random numbers 1...50 and finds the minimum element, in separate functions
2	Calculates the recursive function f(x) of variant 4
3	Calculates `Z` at a point with three functions: input, calculation, display

Task 1

A given two-dimensional array of n×t elements. Write a program that, with separate functions, performs actions according to the option:

1) fills the array with random numbers in the range from 1 to 50 and finds the value of the minimum array element.

1.1. Block diagram of the program algorithm

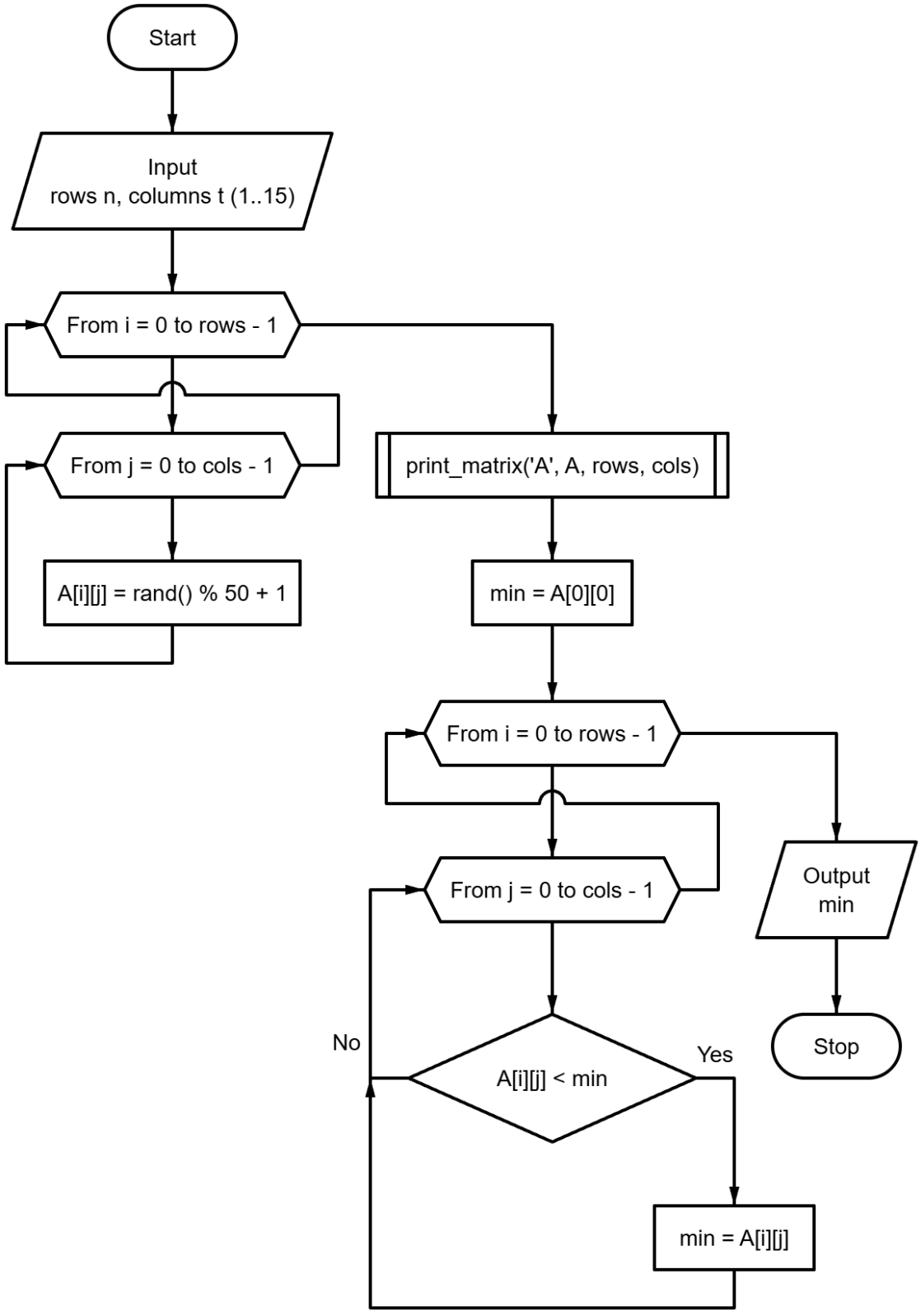


Fig. 1 – Block diagram of the program algorithm

1.2. Program text

```
#include "tasks.h"
#include "utils.h"
#include "matrix.h"

#include <stdio.h>
#include <stdlib.h>

#define RANDOM_MIN 1
#define RANDOM_MAX 50

static void fill_random(
    long a[][MAX_MATRIX_DIM],
    const long rows,
    const long cols
) {
    for (long i = 0; i < rows; ++i) {
        for (long j = 0; j < cols; ++j) {
            const int r = rand(); // NOLINT(*-msc50-cpp)
            a[i][j] = r % (RANDOM_MAX - RANDOM_MIN + 1) +
RANDOM_MIN;
        }
    }
}

static long find_min(
    long a[][MAX_MATRIX_DIM],
    const long rows,
    const long cols
) {
    long min = a[0][0];

    for (long i = 0; i < rows; ++i) {
        for (long j = 0; j < cols; ++j) {
            if (a[i][j] < min) {
                min = a[i][j];
            }
        }
    }

    return min;
}

void run_task1(void) {
    Matrix a;
    long rows = 0;
    long cols = 0;
```

```
    puts("\n=== Task 1: minimum element of a random array
===\n");

    if (!read_matrix_size("the array", &rows, &cols)) {
        return;
    }

    fill_random(a, rows, cols);

    puts("");
    print_matrix("A", a, rows, cols);
    printf("\nMinimum element: %ld\n", find_min(a, rows,
cols));
}
```

1.3. Program results

```
=== Task 1: minimum element of a random array ===

Enter the number of rows of the array: 4
Enter the number of columns of the array: 5

A (4 x 5) =
 2 21  9  2  7
20 11 32 18  8
 9  2 23  2 25
24  7  7 49 50

Minimum element: 2
```

Fig. 2 – Task 1 results

Task 2

Develop an algorithm and program using recursion to calculate function values. The argument of the function belongs to a specific given set of natural numbers.

Table 2 – Idividual task

Variant	Function	Analitiacal expression	Conditions
1	*f(x)*	$\begin{cases} 3 \\ f(x-1)/3 \end{cases}$	$x = 6$ $x > 6$

2.1. Block diagram of the program algorithm

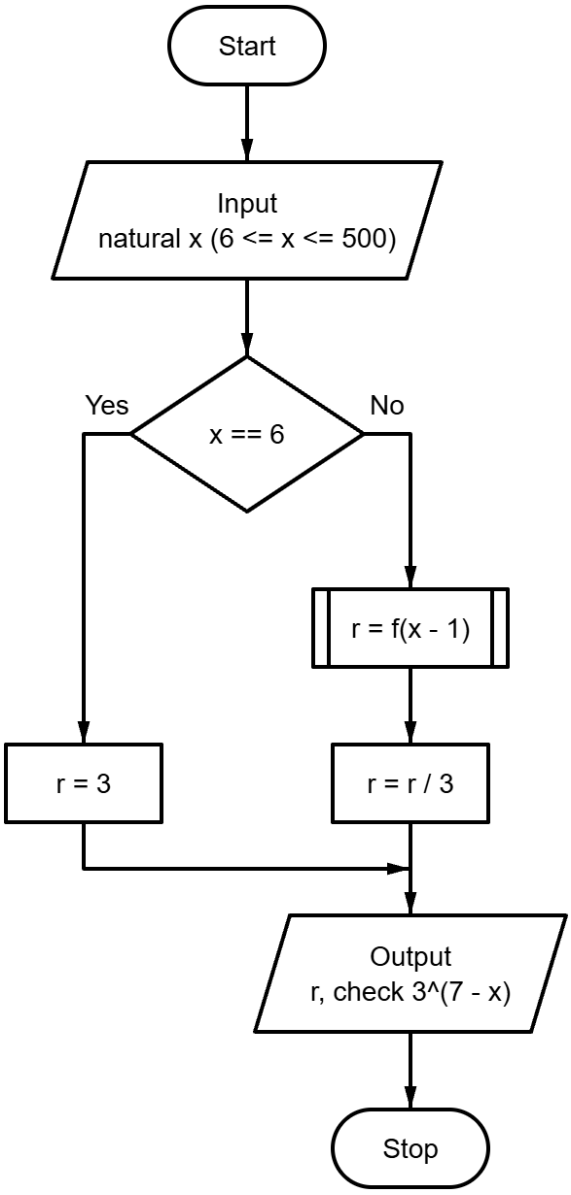


Fig. 3 – Block diagram of the program algorithm

2.2. Program text

```
#include "tasks.h"
#include "utils.h"

#include <math.h>
#include <stdio.h>

#define BASE_ARGUMENT 6
#define BASE_VALUE 3.0
#define DIVISOR 3.0
#define MAX_ARGUMENT 500

static double f(const long x) {
    if (x == BASE_ARGUMENT) {
        return BASE_VALUE;
    }

    return f(x - 1) / DIVISOR;
}

void run_task2(void) {
    long x = 0;

    puts("\n=== Task 2: recursive function ===\n");
    puts("f(x) = 3 for x = 6,  f(x) = f(x - 1) / 3 for x > 6\n");

    if (!read_long_in_range(
            "Enter a natural x (x >= 6): ", BASE_ARGUMENT,
MAX_ARGUMENT, &x
        )) {
        return;
    }

    printf("\nf(%ld) = %.10g\n", x, f(x));

    // f(x) = 3 / 3^(x - 6) = 3^(7 - x)
    printf("Check, 3^(7 - x): %.10g\n", pow(BASE_VALUE, 7 - x));
}
```

2.3. Program results

```
=== Task 2: recursive function ===

f(x) = 3 for x = 6,  f(x) = f(x - 1) / 3 for x > 6

Enter a natural x (x >= 6): 6

f(6) = 3
Check, 3^(7 - x): 3
```

```
=== Task 2: recursive function ===

f(x) = 3 for x = 6,  f(x) = f(x - 1) / 3 for x > 6

Enter a natural x (x >= 6): 8

f(8) = 0.3333333333
Check, 3^(7 - x): 0.3333333333
```

```
=== Task 2: recursive function ===

f(x) = 3 for x = 6,  f(x) = f(x - 1) / 3 for x > 6

Enter a natural x (x >= 6): 10

f(10) = 0.03703703704
Check, 3^(7 - x): 0.03703703704
```

Fig. 4 – Task 2 results

Task 3

Write a program for determining the value of a function at a point. It provides for the implementation of three functions: the first provides input of information, the second - calculation of the required value, the third - display of results. Transfer parameters to organize without using global variables. Task option:

$$Z = \frac{\ln\left(\frac{x}{y}\right)+\sqrt{ax^2+x}+j}{e^{-\frac{x}{y}}\cdot\sin(x^2+y)}, \quad (1)$$

The program is split into three functions, as the assignment requires, and the data goes between them only through parameters, without global variables (see Table 3).

Table 3 – Program internal functions

Function	Does
read_input(&x, &y, &a, &b, &j)	input of the information: the values are returned through pointers (passing by reference)
calculate_z(x, y, a, b, j, &z)	calculation of the required value: the arguments are passed by value, Z through a pointer, the status is returned
print_result(x, y, a, b, j, z)	display of the results

The function is defined not for all values, so “calculate_z” returns a status, and the program reports the reason and asks for the values again (see Table 4).

Table 4 – Calculation statuses

Status	When
CALC_BAD_LOG_ARGUMENT	y = 0 or x / y <= 0, ln(x / y) is undefined
CALC_BAD_SQRT_ARGUMENT	a * x^2 + b < 0
CALC_ZERO_DENOMINATOR	e^(-x / y) * sin(x^2 + y) = 0
CALC_NOT_FINITE	the result does not fit into double

3.1. Block diagram of the program algorithm

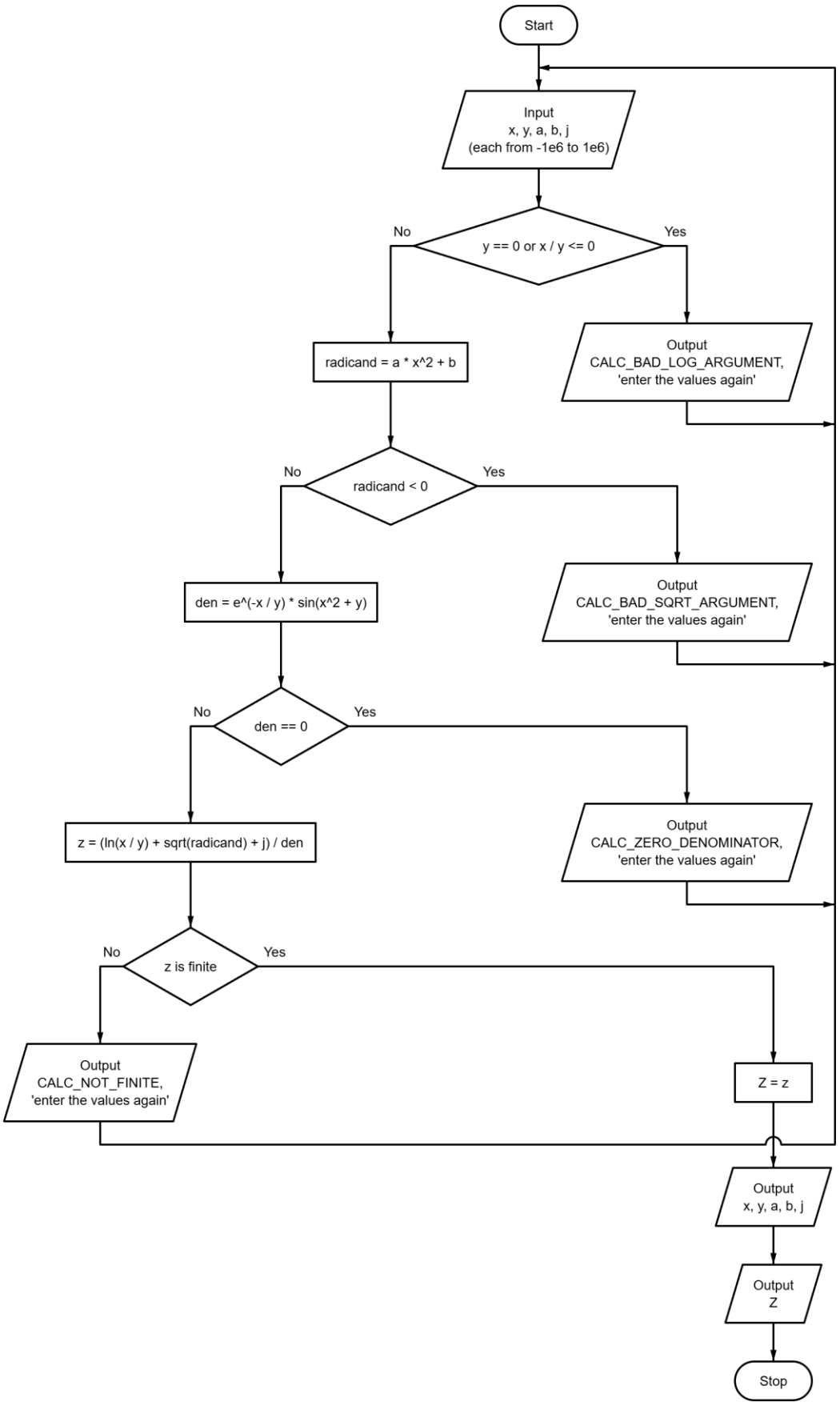


Fig. 5 – Block diagram of the program algorithm

3.2. Program text

```
#include "tasks.h"
#include "utils.h"

#include <math.h>
#include <stdbool.h>
#include <stdio.h>

typedef enum {
    CALC_OK,
    CALC_BAD_LOG_ARGUMENT,
    CALC_BAD_SQRT_ARGUMENT,
    CALC_ZERO_DENOMINATOR,
    CALC_NOT_FINITE
} CalcStatus;

static bool read_input(
    double *const x,
    double *const y,
    double *const a,
    double *const b,
    double *const j
) {
    return read_double_in_range(
                "Enter x: ", -MAX_INPUT_ABS, MAX_INPUT_ABS, x
            ) &&
            read_double_in_range(
                "Enter y: ", -MAX_INPUT_ABS, MAX_INPUT_ABS, y
            ) &&
            read_double_in_range(
                "Enter a: ", -MAX_INPUT_ABS, MAX_INPUT_ABS, a
            ) &&
            read_double_in_range(
                "Enter b: ", -MAX_INPUT_ABS, MAX_INPUT_ABS, b
            ) &&
            read_double_in_range("Enter j: ", -MAX_INPUT_ABS,
MAX_INPUT_ABS, j);
}

static CalcStatus calculate_z(
    const double x,
    const double y,
    const double a,
    const double b,
    const double j,
    double *const out_z
) {
```

```
    if (y == 0.0 || x / y <= 0.0) {
        return CALC_BAD_LOG_ARGUMENT;
    }

    const double radicand = a * x * x + b;

    if (radicand < 0.0) {
        return CALC_BAD_SQRT_ARGUMENT;
    }

    const double denominator = exp(-x / y) * sin(x * x + y);

    if (denominator == 0.0) {
        return CALC_ZERO_DENOMINATOR;
    }

    const double z = (log(x / y) + sqrt(radicand) + j) /
denominator;

    if (!isfinite(z)) {
        return CALC_NOT_FINITE;
    }

    *out_z = z;

    return CALC_OK;
}

static void print_error(const CalcStatus status) {
    switch (status) {
        case CALC_BAD_LOG_ARGUMENT:
            puts(
                "Error: x / y must be positive (y != 0), ln(x / y)
is "
                "undefined."
            );
            break;
        case CALC_BAD_SQRT_ARGUMENT:
            puts(
                "Error: a * x^2 + b must not be negative under the
square root."
            );
            break;
        case CALC_ZERO_DENOMINATOR:
            puts("Error: the denominator e^(-x / y) * sin(x^2 + y)
is zero.");
            break;
```

```
        case CALC_NOT_FINITE:
            puts("Error: the result is too large to be
represented.");
            break;
        case CALC_OK: break;
    }
}

static void print_result(
    const double x,
    const double y,
    const double a,
    const double b,
    const double j,
    const double z
) {
    printf("\nx = %g, y = %g, a = %g, b = %g, j = %g\n", x, y, a,
b, j);
    printf("Z = %.10g\n", z);
}

void run_task3(void) {
    double x = 0.0;
    double y = 0.0;
    double a = 0.0;
    double b = 0.0;
    double j = 0.0;
    double z = 0.0;

    puts("\n=== Task 3: value of a function at a point ===\n");
    puts("        ln(x / y) + sqrt(a * x^2 + b) + j");
    puts("Z = ------------------------------------");
    puts("          e^(-x / y) * sin(x^2 + y)\n");

    while (true) {
        if (!read_input(&x, &y, &a, &b, &j)) {
            return;
        }

        const CalcStatus status = calculate_z(x, y, a, b, j, &z);

        if (status == CALC_OK) {
            break;
        }

        print_error(status);
        puts("Please enter the values again.\n");
```

```
    }

    print_result(x, y, a, b, j, z);
}
```

3.3. Program results

```
=== Task 3: value of a function at a point ===

        ln(x / y) + sqrt(a * x^2 + b) + j
Z = ------------------------------------
          e^(-x / y) * sin(x^2 + y)

Enter x: 2
Enter y: 1
Enter a: 1
Enter b: 1
Enter j: 1

x = 2, y = 1, a = 1, b = 1, j = 1
Z = -30.27683415
```

```
=== Task 3: value of a function at a point ===

        ln(x / y) + sqrt(a * x^2 + b) + j
Z = ------------------------------------
          e^(-x / y) * sin(x^2 + y)

Enter x: 1
Enter y: 0
Enter a: 1
Enter b: 1
Enter j: 1
Error: x / y must be positive (y != 0), ln(x / y) is undefined.
Please enter the values again.
```

Fig. 6 – Task 3 results

Conclusions

During the execution of this laboratory work, practical skills were acquired in designing, declaring, and applying user-defined functions in C11. The tasks demonstrated effective functional decomposition across various computational problems, including matrix processing through pseudorandom array generation ($n \times t$) and minimum element searching in dedicated modules, recursive function evaluation $f(x)$, and decoupled workflow modularization (separating user input retrieval, core mathematical calculation of Z, and formatted output display).

Furthermore, experience was gained in building cross-platform modular C software structured with CMake and Ninja, enforcing strict compiler diagnostics, and implementing robust input validation within an interactive task execution engine.

In Table 5, all results of the completed tasks are highlighted.

Table 5 – Results overview

Task	Input	Output
1	rows = 4, cols = 5	a random 4 x 5 array with numbers 1...50 and its minimum element
2	x = 6	f(6) = 3
	x = 8	f(8) = 0.3333333333
	x = 10	f(10) = 0.03703703704
3	x = 2, y = 1, a = 1, b = 1, j = 1	Z = -30.27683415
	x = 1, y = 0	x / y must be positive, the values are asked again

APPENDIX A

Block diagram of the main program control flow algorithm

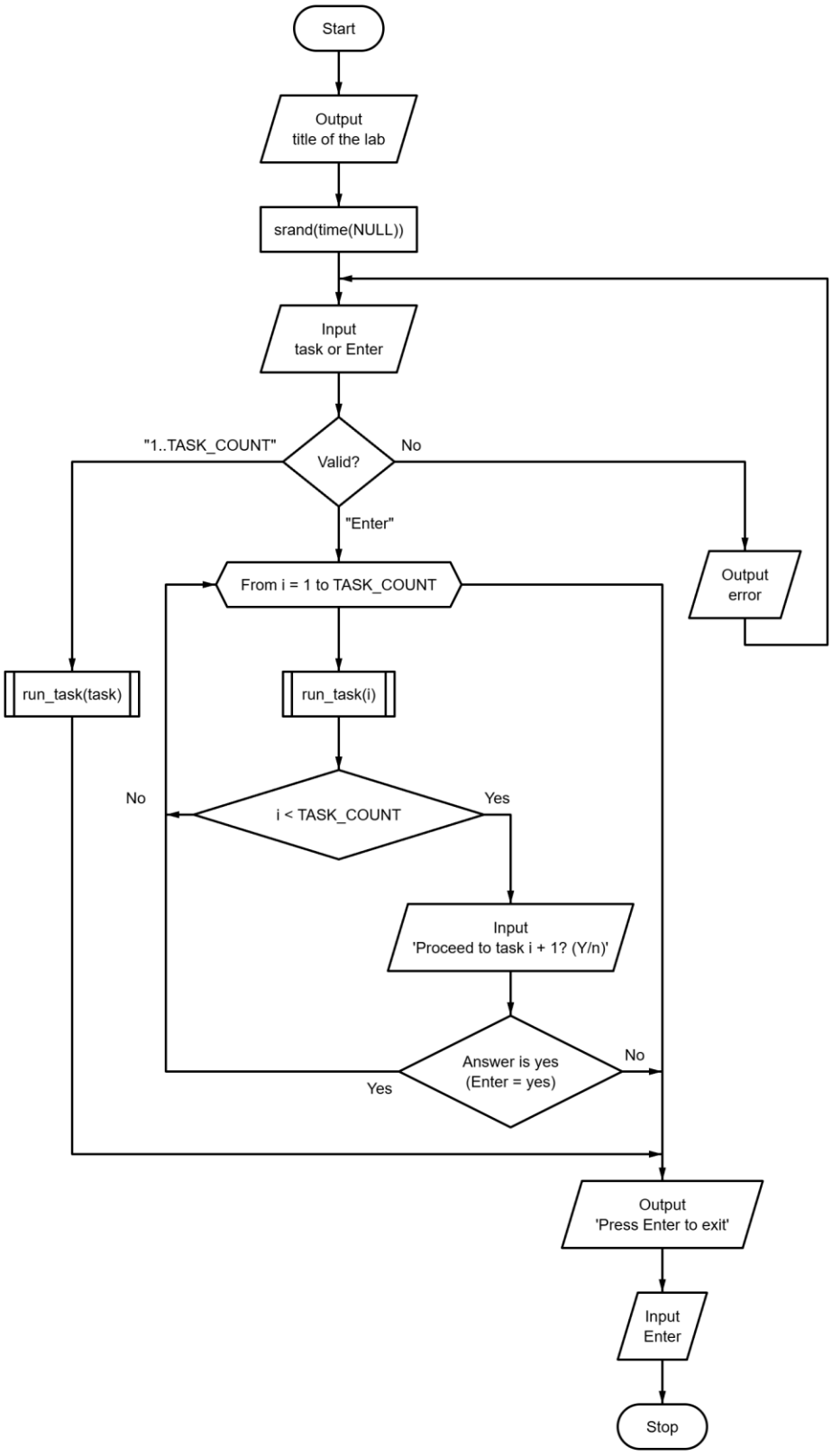


Fig. A.1 – Block diagram of the main program control flow algorithm

APPENDIX B

Source code of the “tasks.h” file

```
#ifndef TASKS_H
#define TASKS_H

#define TASK_LIST X(1) X(2) X(3)

#define X(num) void run_task##num(void);
TASK_LIST
#undef X

#define X(num) +1
enum { TASK_COUNT = 0 TASK_LIST };
#undef X

void run_task(int n);
void run_all_tasks(void);

#endif // TASKS_H
```

APPENDIX C

Source code of the “matrix.h” file

```
#ifndef MATRIX_H
#define MATRIX_H

#include <stdbool.h>

/* Max size of a prompt for user input */
#define PROMPT_SIZE 96

/* Max number of rows and columns of a matrix */
#define MAX_MATRIX_DIM 15

typedef long Matrix[MAX_MATRIX_DIM][MAX_MATRIX_DIM];

bool read_matrix_size(const char *name, long *out_rows, long
*out_cols);
void print_matrix(const char *name, Matrix matrix, long rows, long
cols);

#endif // MATRIX_H
```

APPENDIX D

Source code of the “utils.h” file

```
#ifndef UTILS_H
#define UTILS_H

#include <stdbool.h>
#include <stddef.h>

// Max absolute value of a real number accepted from the keyboard
#define MAX_INPUT_ABS 1e6

bool read_double(const char *prompt, double *out_value);
bool read_doubles(const char *prompt, size_t count, ...);
bool read_long(const char *prompt, long *out_value);
bool read_longs(const char *prompt, size_t count, ...);

bool read_long_in_range(
    const char *prompt,
    long min,
    long max,
    long *out_value
);
bool read_optional_long_in_range(
    const char *prompt,
    long min,
    long max,
    long *out_value,
    bool *out_provided
);
bool read_double_in_range(
    const char *prompt,
    double min,
    double max,
    double *out_value
);

bool confirm(const char *prompt, bool default_answer);

#endif // UTILS_H
```

APPENDIX E

Source code of the “main.c” file

```
#include "tasks.h"
#include "utils.h"
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define PROMPT_SIZE 96

int main(void) {
    puts(
        "========================================\n"
        " Laboratory work 6: Functions (variant 9) \n"
        "========================================\n"
    );

    srand((unsigned)time(NULL)); // NOLINT(*-msc51-cpp)

    char prompt[PROMPT_SIZE];
    snprintf(
        prompt, sizeof(prompt),
        "Enter task number (1-%d) or press Enter to run all tasks
[ALL]: ", TASK_COUNT
    );

    long task = 0;
    bool provided = false;

    if (!read_optional_long_in_range(prompt, 1, TASK_COUNT, &task,
&provided)) {
        return 1;
    }

    if (provided) {
        run_task((int)task);
    } else {
        run_all_tasks();
    }

    printf("\nPress Enter to exit...");
    getchar();

    return 0;
}
```

APPENDIX F

Source code of the “runner.c” file

```
#include "tasks.h"
#include "utils.h"

#include <stdio.h>

static void (*const TASKS[TASK_COUNT])(void) = {
#define X(num) run_task##num,
    TASK_LIST
#undef X
};

void run_task(const int n) {
    TASKS[n - 1]();
}

void run_all_tasks(void) {
    for (int i = 1; i <= TASK_COUNT; ++i) {
        run_task(i);

        if (i < TASK_COUNT) {
            char prompt[64];
            snprintf(
                prompt, sizeof(prompt), "\nProceed to task %d?
(Y/n): ", i + 1
            );

            if (!confirm(prompt, true)) {
                break;
            }
        }
    }
}
```

APPENDIX G

Source code of the “matrix.c” file

```
#include "matrix.h"

#include "utils.h"

#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

bool read_matrix_size(
    const char *const name,
    long *const out_rows,
    long *const out_cols
) {
    char prompt[PROMPT_SIZE];

    snprintf(prompt, sizeof(prompt), "Enter the number of rows of
%s: ", name);

    if (!read_long_in_range(prompt, 1, MAX_MATRIX_DIM, out_rows))
{
        return false;
    }

    snprintf(
        prompt, sizeof(prompt), "Enter the number of columns of
%s: ", name
    );

    return read_long_in_range(prompt, 1, MAX_MATRIX_DIM,
out_cols);
}

void print_matrix(
    const char *const name,
    Matrix matrix,
    const long rows,
    const long cols
) {
    char text[32];
    size_t width = 1;

    for (long i = 0; i < rows; ++i) {
        for (long j = 0; j < cols; ++j) {
```

```
            snprintf(text, sizeof(text), "%ld", matrix[i][j]);

            if (strlen(text) > width) {
                width = strlen(text);
            }
        }
    }

    printf("%s (%ld x %ld) =\n", name, rows, cols);

    for (long i = 0; i < rows; ++i) {
        for (long j = 0; j < cols; ++j) {
            printf(j == 0 ? "%*ld" : " %*ld", (int)width,
matrix[i][j]);
        }

        puts("");
    }
}
```

APPENDIX H

Source code of the “utils.c” file

```
#include "utils.h"

#include <ctype.h>
#include <errno.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

enum { MAX_READ_VALUES = 8 };

typedef bool (*token_parser_fn)(const char *str, char **endptr,
void *out);

static bool is_buffer_empty(const char *ptr) {
    while (*ptr != '\0') {
        if (!isspace((unsigned char)*ptr)) {
            return false;
        }

        ptr++;
    }

    return true;
}

static bool parse_double_token(const char *str, char **endptr,
void *out) {
    errno = 0;
    *(double *)out = strtod(str, endptr);

    return *endptr != str && errno == 0;
}

static bool parse_long_token(const char *str, char **endptr, void
*out) {
    errno = 0;
    *(long *)out = strtol(str, endptr, 10);

    return *endptr != str && errno == 0;
}

static bool read_line_values(
    const char *prompt,
```

```
    const size_t count,
    void *const outs[],
    const token_parser_fn parse
) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    if (prompt != NULL) {
        printf("%s", prompt);
    }

    char buffer[256];
    if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
        return false;
    }

    buffer[strcspn(buffer, "\r\n")] = '\0';

    const char *ptr = buffer;

    for (size_t i = 0; i < count; ++i) {
        if (outs[i] == NULL) {
            return false;
        }

        char *endptr;
        if (!parse(ptr, &endptr, outs[i])) {
            return false;
        }

        ptr = endptr;
    }

    return is_buffer_empty(ptr);
}

bool read_double(const char *prompt, double *out_value) {
    return read_doubles(prompt, 1, out_value);
}

bool read_doubles(const char *prompt, const size_t count, ...) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    void *outs[MAX_READ_VALUES];
```

```
    va_list args;
    va_start(args, count);

    for (size_t i = 0; i < count; ++i) {
        outs[i] = va_arg(args, double *);
    }

    va_end(args);

    return read_line_values(prompt, count, outs,
parse_double_token);
}

bool read_long(const char *prompt, long *out_value) {
    return read_longs(prompt, 1, out_value);
}

bool read_longs(const char *prompt, const size_t count, ...) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    void *outs[MAX_READ_VALUES];

    va_list args;
    va_start(args, count);

    for (size_t i = 0; i < count; ++i) {
        outs[i] = va_arg(args, long *);
    }

    va_end(args);

    return read_line_values(prompt, count, outs,
parse_long_token);
}

bool read_long_in_range(
    const char *prompt,
    const long min,
    const long max,
    long *out_value
) {
    while (true) {
        if (read_long(prompt, out_value)) {
            if (*out_value >= min && *out_value <= max) {
```

```
                return true;
            }

            printf("Error: value must be between %ld and %ld.\n",
min, max);
        } else if (feof(stdin)) {
            return false;
        } else {
            puts("Invalid input. Please enter an integer.");
        }
    }
}

bool read_double_in_range(
    const char *prompt,
    const double min,
    const double max,
    double *out_value
) {
    while (true) {
        if (read_double(prompt, out_value)) {
            // Written this way so that NaN is rejected as well
            if (*out_value >= min && *out_value <= max) {
                return true;
            }

            printf("Error: value must be between %g and %g.\n",
min, max);
        } else if (feof(stdin)) {
            return false;
        } else {
            puts("Invalid input. Please enter a number.");
        }
    }
}

bool read_optional_long_in_range(
    const char *prompt,
    const long min,
    const long max,
    long *out_value,
    bool *out_provided
) {
    char buffer[256];

    while (true) {
        printf("%s", prompt);
```

```
        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            return false;
        }

        if (is_buffer_empty(buffer)) {
            *out_provided = false;
            return true;
        }

        char *endptr;
        long value;

        if (!parse_long_token(buffer, &endptr, &value) ||
            !is_buffer_empty(endptr)) {
            puts("Invalid input. Please enter an integer or press
Enter.");
        } else if (value < min || value > max) {
            printf("Error: value must be between %ld and %ld.\n",
min, max);
        } else {
            *out_value = value;
            *out_provided = true;
            return true;
        }
    }
}

static bool equals_ignore_case(
    const char *str,
    const size_t len,
    const char *word
) {
    if (strlen(word) != len) {
        return false;
    }

    for (size_t i = 0; i < len; ++i) {
        if (tolower((unsigned char)str[i]) != word[i]) {
            return false;
        }
    }

    return true;
}

bool confirm(const char *prompt, const bool default_answer) {
```

```
    char buffer[16];

    while (true) {
        if (prompt != NULL) {
            printf("%s", prompt);
        }

        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            return false;
        }

        const bool overflow = strchr(buffer, '\n') == NULL &&
!feof(stdin);
        if (overflow) {
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF)
                ;
        } else {
            const char *start = buffer;
            while (isspace((unsigned char)*start)) {
                start++;
            }

            size_t len = strlen(start);
            while (len > 0 && isspace((unsigned char)start[len -
1])) {
                len--;
            }

            if (len == 0) {
                return default_answer;
            }

            if (equals_ignore_case(start, len, "y") ||
                equals_ignore_case(start, len, "yes")) {
                return true;
            }

            if (equals_ignore_case(start, len, "n") ||
                equals_ignore_case(start, len, "no")) {
                return false;
            }
        }

        printf("Please answer 'y' or 'n'.\n");
    }
}
```